

Detecção de Comunidades Críticas de Atendimento ao Cliente em Redes Sociais via Busca pelo Clique Máximo

Matheus Garcia¹, Lucas Henrique¹

¹Departamento de Ciência da Computação — Universidade Federal de Roraima (UFRR)
Boa Vista — RR — Brasil

matheusgarciasam@gmail.com, lucas.henrique.vr@gmail.com

Abstract. *This work presents a combinatorial search engine for detecting critical customer-support communities in social networks. The interactions among users are modeled as an undirected graph and the problem of finding densely connected groups (crisis cohorts) is reduced to the Maximum Clique problem, which is NP-hard. Two heuristics were implemented in C11 — a Greedy constructive algorithm and a GRASP metaheuristic — both supported by a bitset representation that reduces neighborhood intersection to a bitwise AND. We report a complexity analysis, a theoretical comparison with the exact Bron-Kerbosch algorithm, and an experimental evaluation over DIMACS benchmarks and a real Twitter customer-support dataset.*

Resumo. *Este trabalho apresenta um motor de busca combinatória para a detecção de comunidades críticas de atendimento ao cliente em redes sociais. As interações entre usuários são modeladas como um grafo não-direcionado e o problema de encontrar grupos densamente conectados (coortes de crise) é reduzido ao problema do Clique Máximo, NP-difícil. Foram implementadas duas heurísticas em C11 — um algoritmo Guloso construtivo e uma metaheurística GRASP — ambas apoiadas por uma representação em bitset que reduz a interseção de vizinhanças a um AND bit a bit. Apresentamos a análise de complexidade, uma comparação teórica com o algoritmo exato de Bron-Kerbosch e uma avaliação experimental sobre benchmarks DIMACS e um conjunto real de tweets de atendimento ao cliente.*

1. Introdução e Cenário de Aplicação

Em plataformas de redes sociais, empresas mantêm canais de atendimento ao cliente que recebem grande volume de interações diárias. Quando ocorre uma falha sistêmica — a indisponibilidade de um serviço, uma cobrança indevida em massa ou um defeito de produto — surge rapidamente um conjunto de usuários *altamente interconectados*, todos reagindo ao mesmo problema, respondendo uns aos outros e mencionando-se mutuamente. Esses agrupamentos, aqui chamados de *coortes de crise*, são valiosos para a operação de suporte: identificá-los permite priorizar incidentes sistêmicos, dimensionar equipes e medir o alcance de uma falha.

O problema de localizar o maior grupo em que *todos* os usuários interagiram entre si corresponde, em teoria dos grafos, ao problema do **Clique Máximo**. Modelando a rede como um grafo não-direcionado $G = (V, E)$, em que cada vértice é um usuário e cada aresta uma interação (resposta direta ou menção), uma coorte de crise é um subconjunto

$C \subseteq V$ tal que todo par $(u, v) \in C$ possui aresta direta, e busca-se maximizar $|C|$. O Clique Máximo é um problema NP-difícil [1], o que motiva o emprego de heurísticas eficientes para instâncias de larga escala.

Este relatório descreve a formulação do problema (Seção 2), os algoritmos implementados (Seção 3), a análise de complexidade com comparação teórica ao algoritmo exato de Bron–Kerbosch (Seção 4), os detalhes de implementação (Seção 5), o estudo comparativo de desempenho (Seção 6) e a aplicação prática sobre dados reais do Twitter (Seção 7).

2. Definição Formal do Problema

Seja $G = (V, E)$ um grafo não-direcionado simples, com $n = |V|$ vértices e $m = |E|$ arestas. Um **clique** é um subconjunto $C \subseteq V$ tal que $\forall u, v \in C, u \neq v \Rightarrow (u, v) \in E$, isto é, o subgrafo induzido por C é completo. O **Clique Máximo** é o clique de maior cardinalidade em G , e seu tamanho é o *número de clique* $\omega(G)$.

$$\omega(G) = \max\{|C| : C \subseteq V \text{ e } C \text{ é clique}\}. \quad (1)$$

A versão de decisão (“existe clique de tamanho $\geq k$?”) é NP-completa, sendo um dos 21 problemas clássicos de Karp; a versão de otimização é NP-difícil. Não se conhece algoritmo polinomial exato, e o problema é, ainda, difícil de aproximar.

3. Algoritmos Implementados

3.1. Representação do Grafo

A escolha da estrutura de dados é determinante para o desempenho da busca por cliques, cuja operação central é a *interseção de vizinhanças*. O grafo é representado de forma **dupla e simultânea**:

- **Bitset (vetor de bits)** — estrutura principal. Cada vértice i possui um vetor de palavras `uint64_t` no qual o bit j indica a adjacência com o vértice j . A interseção de duas vizinhanças — coração da expansão de cliques — reduz-se a um *AND bit a bit*, percorrendo $\lceil n/64 \rceil$ palavras em vez de n posições. A contagem de candidatos usa a instrução de hardware `popcount`.
- **Lista de adjacência** — estrutura auxiliar para percurso geral de vizinhos e navegação esparsa.

3.2. Módulo 1 — Algoritmo Guloso

A heurística gulosa constrói um único clique de forma determinística:

1. Seleciona como semente o vértice de *maior grau global*.
2. Mantém um conjunto de candidatos P (em bitset) com os vértices adjacentes a todos os membros já no clique.
3. A cada passo, escolhe de P o candidato com *maior grau dentro de P* (maior número de vizinhos comuns), maximizando a chance de continuar a expansão.
4. Adiciona o candidato ao clique e atualiza $P \leftarrow P \cap N(v)$ via AND de bitsets.
5. Termina quando $P = \emptyset$.

O algoritmo é rápido, porém míope: por nunca revisar escolhas, fica preso em ótimos locais.

3.3. Módulo 2 — Metaheurística GRASP

O GRASP (*Greedy Randomized Adaptive Search Procedure*) executa `max_iter` iterações independentes, retendo o melhor clique encontrado. Cada iteração combina duas fases:

Fase construtiva (RCL). Idêntica ao guloso, exceto que, em vez de sempre tomar o melhor candidato, sorteia aleatoriamente entre os `rcl_size` melhores (a *Restricted Candidate List*). Isso diversifica as soluções entre iterações. A aleatoriedade usa um gerador congruencial linear (LCG) com semente fixa, garantindo reprodutibilidade.

Busca local 1-opt. Tenta substituir um vértice do clique por *dois* vértices externos mutuamente adjacentes e adjacentes a todos os demais membros. Cada troca bem-sucedida aumenta a cardinalidade em uma unidade; o processo repete enquanto houver melhoria.

A combinação de construção aleatorizada com refinamento local permite ao GRASP escapar dos ótimos locais que limitam o guloso.

4. Análise de Complexidade

Seja $n = |V|$, $m = |E|$, ω o tamanho do clique encontrado e $W = \lceil n/64 \rceil$ o número de palavras de um bitset. A Tabela 1 resume as operações fundamentais.

Tabela 1. Complexidade das operações e algoritmos.

Operação / Algoritmo	Complexidade
AND de bitsets ($P \cap N(v)$)	$O(n/64)$
popcount de bitset	$O(n/64)$
Inserção de aresta	$O(1)$
Teste de aresta	$O(1)$
Leitor DIMACS	$O(n + m)$
Parser Twitter (CSV)	$O(L)$ (L = linhas)
Guloso	$O(\omega \cdot n^2/64)$
GRASP	$O(\text{max_iter} \cdot \omega \cdot n^2/64)$

Guloso. A busca da semente custa $O(n)$. O laço principal executa até ω passos; em cada passo varre os candidatos de P (até n) e, para cada um, realiza um AND e um popcount de custo $O(n/64)$. Logo, cada passo custa $O(n \cdot n/64) = O(n^2/64)$ e o total é $O(\omega \cdot n^2/64)$. Como $\omega \leq n$, o pior caso é $O(n^3/64)$, mas na prática $\omega \ll n$.

GRASP. Cada iteração reproduz o custo da construção gulosa, acrescido de ordenações $O(n \log n)$ por passo (para montar a RCL) e da busca local, cuja varredura custa $O(\omega \cdot n^2/64)$ por passada. Multiplicando pelas `max_iter` iterações, obtém-se $O(\text{max_iter} \cdot \omega \cdot n^2/64)$. O fator $1/64$ proveniente do bitset é a principal vantagem prática da implementação.

4.1. Comparação Teórica com o Algoritmo Exato

Por ser NP-difícil, o Clique Máximo não admite solução exata polinomial conhecida. Para situar as heurísticas, consideramos dois métodos exatos de referência.

Força bruta. Enumerar todos os 2^n subconjuntos de V e verificar, em $O(n^2)$, se cada um é clique resulta em $O(2^n \cdot n^2)$. É inviável já para $n \approx 40$.

Bron–Kerbosch. O algoritmo de Bron–Kerbosch [2] enumera *todos* os cliques maximais por *backtracking* recursivo sobre três conjuntos: R (clique parcial), P (candidatos) e X (já explorados):

```
BronKerbosch(R, P, X):  
  se P = vazio e X = vazio:  
    reporta R como clique maximal  
  para cada v em P:  
    BronKerbosch(R + {v}, P inter N(v), X inter N(v))  
  P <- P \ {v}  
  X <- X + {v}
```

Com a escolha de um *pivô* para reduzir a ramificação, sua complexidade de pior caso é $O(3^{n/3})$, que é ótima no sentido de Moon–Moser: um grafo de n vértices pode ter até $3^{n/3}$ cliques maximais. Com ordenação por *degeneracy* d , obtém-se $O(d \cdot n \cdot 3^{d/3})$, eficiente para grafos esparsos. O Clique Máximo seria então o maior clique maximal reportado.

A Tabela 2 contrasta as abordagens. Enquanto Bron–Kerbosch *garante* a solução ótima, seu custo exponencial o restringe a instâncias pequenas; as heurísticas trocam essa garantia por escalabilidade polinomial.

Tabela 2. Heurísticas vs. algoritmo exato.

Método	Complexidade	Garante ótimo?
Força bruta	$O(2^n \cdot n^2)$	Sim
Bron–Kerbosch	$O(3^{n/3})$	Sim
Guloso	$O(\omega \cdot n^2/64)$	Não
GRASP	$O(\text{max_iter} \cdot \omega \cdot n^2/64)$	Não

5. Detalhes de Implementação

O projeto foi desenvolvido inteiramente em **C (padrão C11)**, sem bibliotecas externas de manipulação de grafos, e organizado em módulos coesos:

- `graph.[ch]` — estrutura do grafo (bitset + lista de adjacência), criação, inserção/teste de arestas e estatísticas.
- `clique.[ch]` — algoritmos Guloso e GRASP, e a estrutura `CliqueResult` (vértices, tamanho, iterações, tempo).
- `dimacs.[ch]` — leitor de benchmarks no formato DIMACS `.clq` (linhas `p edge e e u v`).

- `twitter_parser.[ch]` — parser de CSV do dataset Twitter, com tabela `hash string→int` (*djb2*) para mapear identificadores e resolver as arestas de resposta em passagem única.
- `dot_export.[ch]` — exportação para o formato GraphViz `.dot`, destacando o clique em vermelho.
- `main.c` — ponto de entrada com os modos `test`, `greedy`, `grasp`, `twitter` e `bench`.

A compilação usa `make` com as flags `-Wall -Wextra -O2 -std=c11`; o projeto compila sem nenhum aviso. A gestão de memória é explícita, com liberação de todas as estruturas (`graph_free`, `clique_result_free`, `hashmap_free`).

6. Estudo Comparativo de Desempenho

6.1. Metodologia

Os experimentos usaram dois benchmarks padrão da competição DIMACS, amplamente empregados na literatura de Clique Máximo. O GRASP foi configurado com `max_iter = 100`, `ocl_size = 3` e `semente = 42`. A qualidade é medida como a razão entre o clique obtido e o ótimo conhecido. Os tempos foram obtidos via `clock()` (CPU time), com binário compilado em `-O2`. A Tabela 3 caracteriza as instâncias.

Tabela 3. Características das instâncias DIMACS.

Instância	Vért.	Arestas	Densidade	Grau máx.	ω
brock200_2	200	9876	0,4963	114	12
p_hat300-1	300	10933	0,2438	132	8

6.2. Resultados

A Tabela 4 apresenta os resultados. O número de arestas lido coincidiu exatamente com o esperado em ambos os arquivos, validando o leitor DIMACS.

Tabela 4. Comparativo Guloso vs. GRASP nos benchmarks DIMACS.

Instância	Algoritmo	Clique	Ótimo	Qualidade	Tempo (ms)
brock200_2	Guloso	9	12	75,0 %	0,39
brock200_2	GRASP	10	12	83,3 %	49,85
p_hat300-1	Guloso	7	8	87,5 %	0,34
p_hat300-1	GRASP	8	8	100,0 %	42,04

6.3. Discussão

Os resultados confirmam o *trade-off* esperado entre qualidade e tempo:

- O **GRASP supera o Guloso** em ambas as instâncias, atingindo o **ótimo** ($\omega = 8$) em `p_hat300-1` e ficando a apenas dois vértices do ótimo em `brock200_2`, instância notoriamente difícil.
- O custo é maior: o GRASP leva $\sim 40\text{--}50$ ms contra $\sim 0,3$ ms do Guloso (cerca de $100\times$), coerente com o fator `max_iter = 100`.

- O Guloso é apropriado para uma primeira estimativa em tempo quase instantâneo; o GRASP é preferível quando a qualidade da solução é prioritária e o orçamento de tempo permite.

A diferença em `brock200_2` ilustra a dificuldade da família *brock*, projetada com cliques “escondidos” que resistem a heurísticas locais; mais iterações ou uma busca local mais agressiva (2-opt) poderiam reduzir essa lacuna.

7. Aplicação Prática: Dataset Twitter

Para demonstrar a aplicação no cenário-alvo, o módulo Twitter processou uma amostra real do dataset *Customer Support on Twitter* (Kaggle). O parser constrói o grafo a partir do campo `in_response_to_tweet_id`, ligando autores que respondem uns aos outros. Na amostra de 99 tweets, obteve-se um grafo com 44 autores, 28 arestas e densidade 0,0296; a busca gulosa identificou uma coorte (clique) de tamanho 2, compatível com a baixa densidade de uma amostra pequena. Sobre o dataset completo (~ 3 milhões de tweets), a mesma abordagem escala graças à passagem única do parser ($O(L)$) e à representação em bitset. O resultado é exportado em `.dot` e renderizado com GraphViz, destacando a coorte de crise em vermelho.

8. Conclusão

Este trabalho modelou a detecção de comunidades críticas de atendimento ao cliente como o problema do Clique Máximo e implementou, em C11, duas heurísticas — Guloso e GRASP — apoiadas por uma representação em bitset que torna a interseção de vizinhanças um AND bit a bit. A análise de complexidade situou as heurísticas como soluções polinomiais frente ao custo exponencial dos métodos exatos (força bruta e Bron–Kerbosch). Experimentalmente, o GRASP alcançou o ótimo em uma das instâncias e superou consistentemente o Guloso, ao custo de maior tempo de execução. A aplicação ao dataset real do Twitter demonstrou a viabilidade prática da abordagem. Como trabalho futuro, destacam-se a implementação de uma busca local 2-opt, o ajuste adaptativo de `max_iter/rcl_size` e a comparação empírica direta com o Bron–Kerbosch em instâncias pequenas.

Repositório

O código-fonte completo, os benchmarks e as instruções de compilação estão disponíveis em:

https://github.com/Luccas-H/FP_DCC606_Tema_6_RR_2026

Referências

- [1] GAREY, M. R.; JOHNSON, D. S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [2] BRON, C.; KERBOSCH, J. Algorithm 457: Finding All Cliques of an Undirected Graph. *Communications of the ACM*, v. 16, n. 9, p. 575–577, 1973.
- [3] BEHAR, R.; COHEN, S. Finding All Maximal Connected s -Cliques in Social Networks. In: *Proceedings of the 21st International Conference on Extending Database Technology (EDBT)*, p. 61–72, 2018. DOI: 10.5441/002/edbt.2018.07.

- [4] FEO, T. A.; RESENDE, M. G. C. Greedy Randomized Adaptive Search Procedures. *Journal of Global Optimization*, v. 6, p. 109–133, 1995.
- [5] DIMACS Implementation Challenge — Maximum Clique. Disponível em: <http://archive.dimacs.rutgers.edu/Challenges/>.